Apache Spark Internals: How It Works

Document Date: May 26, 2026
Subject: Deep dive into Apache Spark architecture, execution model, and scheduling
Audience: Big Data developers, systems engineers, students

Table of Contents

---

1. Overview

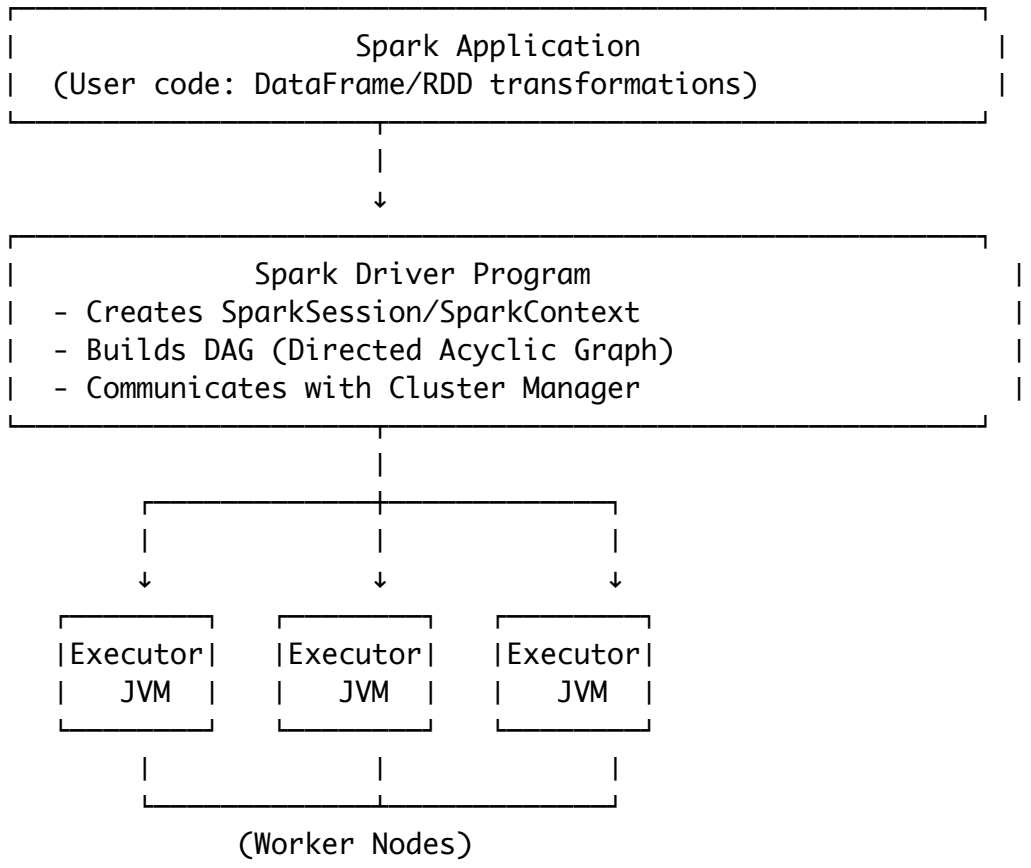
What is Apache Spark?

Apache Spark is a **unified computing engine** for large-scale data processing.

Key Characteristics:

- **Fast:** In-memory computation (10-100× faster than Hadoop MapReduce)
- **Unified:** SQL, streaming, ML, graph processing on single engine
- **Distributed:** Scales across clusters of 10s to 1000s of machines
- **Lazy Evaluation:** Builds execution plans, executes on action
- **Resilient:** RDD (Resilient Distributed Dataset) recovery via lineage

Core Components



---

2. Architecture

2.1 Master-Worker Architecture

Driver Program (Master)

Role: Orchestrates entire computation

Responsibilities:

1. **Parse user code** → Convert to Spark actions
2. **Build DAG** → Logical execution plan
3. **Stage division** → Break DAG at shuffle boundaries
4. **Task creation** → One task per RDD partition
5. **Scheduling** → Assign tasks to executors
6. **Result collection** → Gather outputs from executors

Memory: Typically 1-4 GB (configurable via `spark.driver.memory`)

Process:

```
// Driver runs user code
SparkSession spark = SparkSession.builder()
    .appName("MyApp")
    .getOrCreate();

DataFrame df = spark.read.parquet("data.parquet");
df.filter("age > 30")
  .groupBy("department")
  .agg(sum("salary"))
  .show();  // ← ACTION: Triggers execution

// All above runs in Driver until .show() is called
```

Executors (Workers)

Role: Execute tasks in parallel

Responsibilities:

1. **Receive tasks** from Driver scheduler
2. **Execute task code** on RDD partitions
3. **Store results** in memory (or spill to disk)
4. **Return results** to Driver or write to storage
5. **Communicate** shuffle data between executors

Memory Hierarchy (per executor):

```
Total Memory (e.g., 2GB)
├── Execution Memory (60% = 1.2GB)
|   └── Task computation, joins, sorts
├── Storage Memory (40% = 0.8GB)
|   └── Cache, broadcast variables
└── Reserved/Overhead (fraction automatically reserved)
```

Number of Cores:

- Default: Determined by cluster manager (YARN, Kubernetes)
- Controls task parallelism per executor
- Typical: 2-8 cores per executor

2.2 Cluster Manager Integration

Spark delegates **resource allocation** to cluster managers:

```
Spark Application
      |
      ├─→ YARN (Hadoop) → Allocates containers
      ├─→ Kubernetes → Allocates pods
      ├─→ Mesos → Allocates resources
      ├─→ Standalone → Spark-native cluster manager
      └─→ Local → Single machine, threads as "executors"
```

Your Project: Uses **Local mode** (development/demo)

```
spark = SparkSession.builder \
    .master("local[*]")  # Uses all local cores
    .appName("BDA-Lab")
    .getOrCreate()
```

For Real Clusters:

```
# YARN (Hadoop)
spark = SparkSession.builder \
    .master("yarn") \
    .config("spark.submit.deployMode", "cluster") \
    .getOrCreate()

# Kubernetes
spark = SparkSession.builder \
    .master("k8s://https://kubernetes-cluster:6443") \
    .getOrCreate()
```

2.3 SparkSession vs SparkContext

SparkSession (Modern, Recommended)

Introduced: Spark 2.0+

Features:

- Unified entry point for DataFrames, SQL, Streaming
- Encapsulates SparkContext internally
- Cleaner API

```
# Modern Spark (≥ 2.0)
spark = SparkSession.builder \
    .appName("MyApp") \
    .config("spark.executor.memory", "2g") \
    .getOrCreate()

# Use DataFrame API (SQL-like)
df = spark.read.csv("data.csv", header=True)
df.select("name", "age").show()
```

SparkContext (Legacy)

Legacy: Spark 1.x only

Features:

- Low-level RDD manipulation
- Direct Spark core access

```
# Legacy Spark (1.x only)
sc = SparkContext("local", "MyApp")

# Use RDD API (functional)
rdd = sc.textFile("data.txt")
rdd.map(lambda x: x.split()).collect()
```

In Your Project: Uses SparkSession (correct for Spark 3.x)

---

3. Execution Model

3.1 Lazy Evaluation & Actions vs Transformations

Transformation (Lazy)

Definition: Build up computation graph, don't execute yet

Examples:

```
df.filter("age > 30")           # Filter rows
df.select("name", "age")        # Select columns
df.groupBy("dept").count()      # Group and count
df.join(other_df, "key")        # Join DataFrames
df.map(lambda x: x * 2)         # Map over RDD
```

Execution: None — just updates DAG

Why Lazy?

1. **Optimization:** Spark can fuse transformations
2. **Pruning:** Unused columns eliminated before reading
3. **Efficiency:** User doesn't pay for unused computations

Action (Eager)

Definition: Actually execute and return results to driver or write to storage

Examples:

```
df.show()                          # Print first 20 rows → ACTI
df.collect()                       # Return all rows to driver
df.write.parquet("output/")        # Write to disk → ACTION
df.count()                          # Count rows → ACTION
df.foreach(print)                  # Apply function to each row
df.take(10)                         # Return first 10 rows → AC
df.first()                          # Return first row → ACTION
```

Execution: Yes — builds and executes full DAG

Visual Example

```
# TRANSFORMATIONS (no execution)
df1 = spark.read.csv("input.csv", header=True)     # Read (la
df2 = df1.filter("age > 30")                        # Filter (
df3 = df2.select("name", "salary")                  # Select (
df4 = df3.groupBy("name").agg(sum("salary"))        # GroupBy

# At this point: NO data processed, only DAG built

# ACTION (execution happens here)
result = df4.collect()                                # Trigger

# Now all transformations above execute in sequence
```

DAG Built:

```
Read CSV
    ↓
Filter (age > 30)
    ↓
Select (name, salary)
    ↓
GroupBy (name)
    ↓
Collect (return results)
```

3.2 RDD: Resilient Distributed Dataset

Core abstraction in Spark (underneath DataFrames)

Properties

1. **Resilient:** Recover from node failure via lineage
2. **Distributed:** Partitioned across multiple nodes
3. **Immutable:** Cannot be changed; transformations create new RDDs

Lineage (Fault Tolerance)

```
RDD4 = RDD3.groupBy()
  ↑ (depends on)
RDD3 = RDD2.select()
  ↑ (depends on)
RDD2 = RDD1.filter()
  ↑ (depends on)
RDD1 = read_csv()

// If RDD3 partition is lost, Spark replays:
// RDD1 → filter → RDD2 → select → RDD3
```

Stored Lineage: Entire computation chain stored (metadata only, ~KB)

Recovery: If executor fails, Spark reruns lost stages using lineage

---

DataFrame ≈ RDD + Schema

DataFrame is RDD with **schema** (column names, types)

```
# RDD (untyped)
rdd = sc.textFile("data.txt")  # Just strings
rdd.map(lambda x: x.split())   # User must parse

# DataFrame (typed, optimized)
df = spark.read.csv("data.csv", header=True)  # Schema inferre
df.select("name", "age")  # Type-safe, optimized
```

Why DataFrame Preferred:

- ✓ Optimizer understands structure
- ✓ Faster execution (Catalyst optimizer)
- ✓ SQL interface available
- ✓ Better memory layout (columnar)

3.3 Catalyst Optimizer

What: Spark's **query optimizer** for DataFrames/SQL

Input: Logical execution plan (what user wrote)

Output: Optimized physical plan (what actually runs)

Optimizations:

1. **Predicate Pushdown**

```
# User wrote (bad):
df = spark.read.parquet("huge.parquet")  # Read ALL colum
df = df.filter("region == 'US'")         # Filter after r

# Catalyst rewrites (good):
# Read only rows where region == 'US' at read time
```

2. **Projection Pruning**

```
# User wrote:
df.select("name", "age")  # Only need 2 columns

# Catalyst reads:
# Only name & age columns from source (not all)
```

3. **Constant Folding**

```python
# User wrote:
df.filter("salary > " + str(30000 + 5000))

# Catalyst optimizes:
# df.filter("salary > 35000")  # Compute constant once
```

4. **Join Order**

```python
# Reorder joins to minimize intermediate data:
# df1 (10M rows) join df2 (100K rows) join df3 (50K rows)
# Optimal: (df2 join df3) join df1  → minimize temp data
```

---

4. Scheduler

4.1 Scheduler Hierarchy

```
DAG Scheduler
    |
    ├─→ Builds stages (shuffle boundaries)
    ├─→ Creates task sets
    └─→ Submits to Task Scheduler
         |
         Task Scheduler
         |
         ├─→ Assigns tasks to executors
         ├─→ Handles task retries
         └─→ Manages scheduler backend (local, YARN, Kubernete
         |
         ├─→ Locality Scheduler
         |   └─→ Prefer tasks where data lives (data locality)
         |
         └─→ Scheduling Policies
             ├─→ FIFO (default)
             ├─→ FAIR (multiple queues)
             └─→ [Your project: Adaptive SJF]
```

4.2 DAG Scheduler: Building Stages

Job: Computation triggered by an action

Stage: Set of tasks that can run in parallel (no shuffle needed)

Shuffle: Communication between stages (expensive)

Example: Word Count DAG

```
rdd = sc.textFile("input.txt")          # RDD1
rdd2 = rdd.flatMap(lambda x: x.split()) # RDD2 (no shuffle)
rdd3 = rdd2.map(lambda x: (x, 1))       # RDD3 (no shuffle)
rdd4 = rdd3.reduceByKey(lambda a,b: a+b) # RDD4 (SHUFFLE!)
result = rdd4.collect()                  # ACTION

# DAG:
# Stage 0: Read → FlatMap → Map (can parallelize)
#          |
#          ↓ (shuffle boundary)
#
# Stage 1: ReduceByKey → Collect (can parallelize)
```

Shuffle: Key → value pairs grouped by key across network

Stage Submission:

1. DAGScheduler detects action
2. Builds DAG
3. Identifies stages (shuffle boundaries)
4. Submits Stage 0 to TaskScheduler
5. When Stage 0 completes, submits Stage 1
6. Continues until all stages done

4.3 Task Scheduler: Assigning Tasks to Executors

FIFO Scheduling (Default)

```
Job 1: 10 tasks
Job 2: 5 tasks
Job 3: 2 tasks
Available Executors: 4 cores

Timeline:
[00-05s] Job 1 Tasks 1-4 running on 4 executors
[05-10s] Job 1 Tasks 5-8 running on 4 executors
[10-12s] Job 1 Tasks 9-10 running on 2 executors
         Job 2 Tasks 1-2 running on 2 executors
[12-15s] Job 2 Tasks 3-5 running on 3 executors
[15-17s] Job 3 Tasks 1-2 running on 2 executors
```

Problem: Job 2 & 3 (fast, small) blocked by Job 1 (slow, large)

FAIR Scheduling (Multiple Pools)

```
Configuration (fairscheduler.xml):
Pool A (weight=2): Job 1 (large batch)
Pool B (weight=1): Job 2 (interactive)

Resources: 4 cores

Allocation:
Pool A: 2.67 cores (60%) ≈ 2-3 cores
Pool B: 1.33 cores (40%) ≈ 1-2 cores

Timeline:
[00-05s] Job 1 (A) 2 tasks + Job 2 (B) 2 tasks (4 cores total)
[05-10s] Job 1 (A) 2 tasks + Job 2 (B) 2 tasks
[10-12s] Job 1 (A) 2 tasks + Job 3 (B) 2 tasks (Job 2 done)
```

Benefit: Job 2 starts earlier; interleaved execution

Adaptive/SJF Scheduling (Your Project)

```
Job Queue (sorted by predicted duration):
Job 3: Predicted 2s ← Run first (shortest)
Job 2: Predicted 5s ← Run second
Job 1: Predicted 10s ← Run last (longest)

Resources: 4 cores

Timeline:
[00-02s] Job 3 (4 tasks) → Done!
[02-07s] Job 2 (5 tasks, 4 cores) → Done!
[07-17s] Job 1 (10 tasks, 4 cores) → Done!

Total time: 17s (vs 17s for FIFO, but Job 2 & 3 finish faster!
Avg latency: (2 + 7 + 17) / 3 = 8.67s (vs ~10s for FIFO)
```

4.4 Locality Scheduling

Principle: Run tasks where data lives (minimize network I/O)

Locality Levels (in order of preference):

```
1. PROCESS_LOCAL      (0 ms)   - Task runs in same JVM as datc
2. NODE_LOCAL         (5 ms)   - Task runs on same machine as
3. RACK_LOCAL         (50 ms)  - Task runs in same rack
4. ANY                (200 ms) - Task runs elsewhere
```

Algorithm:

```
For each task:
  1. Check if data exists in PROCESS_LOCAL → Schedule there
  2. Else check NODE_LOCAL → Schedule there
  3. Else check RACK_LOCAL → Schedule there
  4. Else schedule on any available executor (ANY)
  5. If task delayed > 5s waiting for preferred location, sche
```

Why Matters:

- PROCESS_LOCAL vs ANY: **40× faster** (no network)
- Typical network bandwidth: 1 Gbps → 125 MB/s
- Transferring 1 GB of data: 8 seconds (vs 1 ms if local)

Your Project Impact: Single machine → All tasks are PROCESS_LOCAL ✓

---

5. Memory Management

5.1 Memory Layout

Per Executor Memory Allocation:

```
Total Executor Memory (e.g., 2 GB)
|
├─ Reserved Memory (300 MB, fixed)
|  └─ Spark internals overhead
|
└─ Available Memory (1.7 GB) = 2 GB - 300 MB
   |
   ├─ Execution Memory (60% × 1.7 GB = 1.02 GB)
   |  ├─ Shuffle buffers
   |  ├─ Hash aggregation
   |  └─ Sort operations
   |
   └─ Storage Memory (40% × 1.7 GB = 0.68 GB)
      ├─ Cached RDDs / DataFrames
      ├─ Broadcast variables
      └─ Accumulator state
```

Dynamic Allocation:

In Spark 2.0+, memory can be **borrowed** between sections:

- If Storage Memory not full, Execution can use it
- If Execution Memory not full, Storage can use it

- When one section needs space, the other evicts

```
Scenario 1: Heavy caching
┌──────────────────────────┐
| Execution: 200 MB        |
| Storage: 1.5 GB          | ← Borrowed from Execution
└──────────────────────────┘

Scenario 2: Heavy sorting
┌──────────────────────────┐
| Execution: 1.2 GB        | ← Borrowed from Storage
| Storage: 400 MB          |
└──────────────────────────┘
```

5.2 Memory Pressure & Spilling

When memory fills:

```
In-memory data → Execution Memory full?
    |
    ├─ YES → Spill to disk (slow)
    |        - Compress with codec (SNAPPY, LZ4)
    |        - Write to shuffle directory
    |        - Next iteration reads from disk
    |
    └─ NO → Continue in memory (fast)
```

Spill Performance:

- **In-Memory Sort:** 1000x records/sec (millions/sec)
- **Disk Spill:** Limited by disk I/O (10-100x slower)
- **Shuffle Spill:** Network + Disk (slowest operation in Spark)

Your Project Implication:

```
# Small jobs (< 10s) → Fit in memory
df.filter("x > 10").count()  # In-memory ✓

# Large jobs with many partitions → May spill
df = df.repartition(1000)      # Create 1000 partitions
df.groupBy("key").count()      # May cause spill if many uniqu

# Spilling = slow. Reduced parallelism = fast.
```

5.3 Garbage Collection (GC) Pauses

Issue: JVM garbage collection can freeze executor

Typical GC Pause: 100ms - 1s (varies)

Impact on Streaming:

- Streaming jobs have latency SLA (e.g., 5s)
- GC pause → Task delayed → SLA violated

Impact on Your Project:

- Batch jobs tolerate GC (not latency-sensitive)
- But heavy GC increases overall job time
- Example: 1s GC × 10 tasks = 10s lost

Tuning GC:

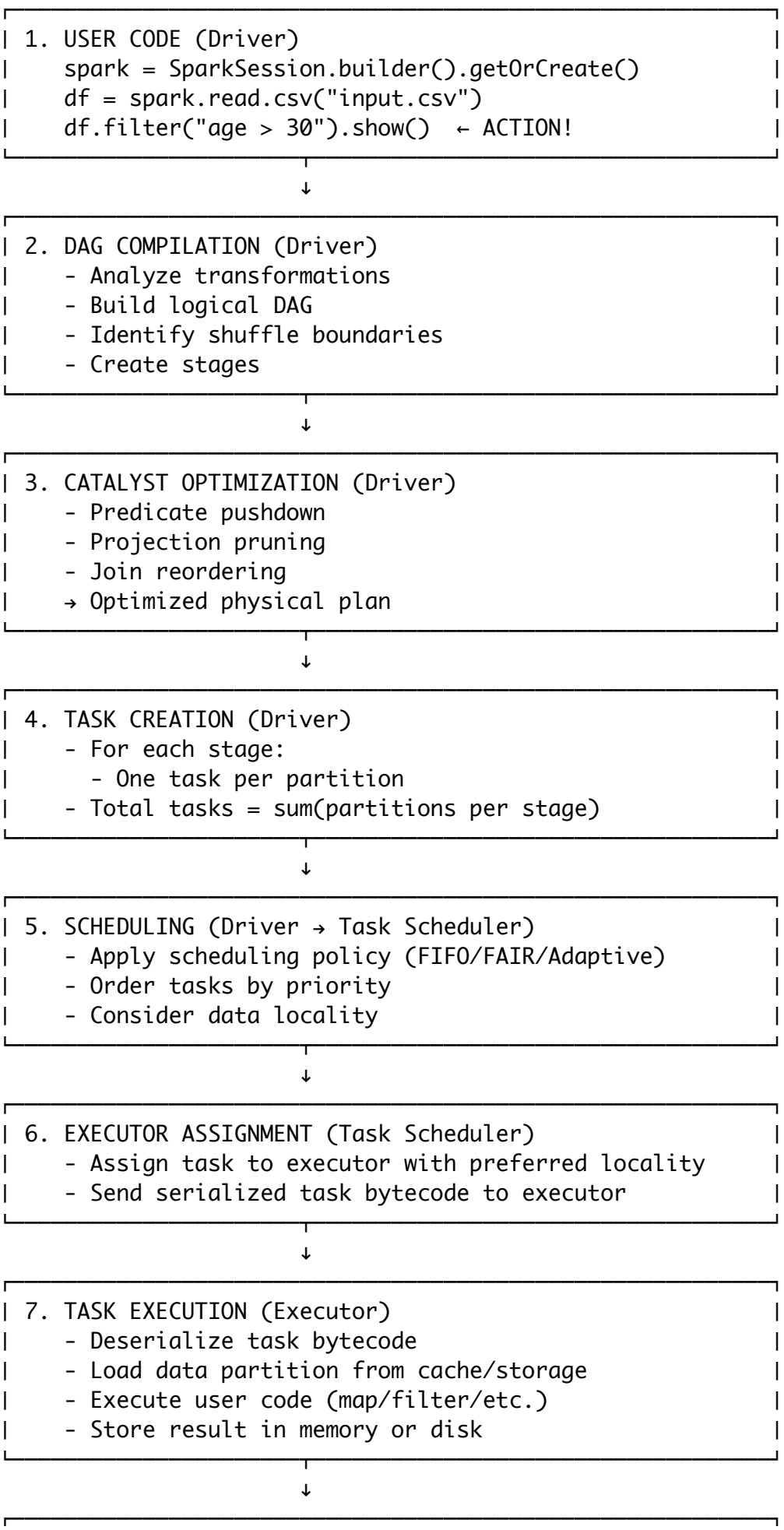
```
# Disable GC overhead (not recommended)
spark-submit \
  --conf spark.executor.extraJavaOptions="-XX:+UseG1GC -XX:Ini
  application.jar

# Typical: G1GC (Garbage First) preferred for large heaps
```

---

6. Job Execution Lifecycle

6.1 Complete Execution Flow


1. USER CODE (Driver)
spark = SparkSession.builder().getOrCreate()
df = spark.read.csv("input.csv")
df.filter("age > 30").show() ← ACTION!
2. DAG COMPILATION (Driver)
- Analyze transformations
- Build logical DAG
- Identify shuffle boundaries
- Create stages
3. CATALYST OPTIMIZATION (Driver)
- Predicate pushdown
- Projection pruning
- Join reordering
→ Optimized physical plan
4. TASK CREATION (Driver)
- For each stage:
- One task per partition
- Total tasks = sum(partitions per stage)
5. SCHEDULING (Driver → Task Scheduler)
- Apply scheduling policy (FIFO/FAIR/Adaptive)
- Order tasks by priority
- Consider data locality
6. EXECUTOR ASSIGNMENT (Task Scheduler)
- Assign task to executor with preferred locality
- Send serialized task bytecode to executor
7. TASK EXECUTION (Executor)
- Deserialize task bytecode
- Load data partition from cache/storage
- Execute user code (map/filter/etc.)
- Store result in memory or disk

```
┌──────────────────────────────────────────────────────────┐
| 8. RESULT COLLECTION (Executor → Driver)                 |
|    - Send task result back to driver                     |
|    - Driver aggregates results                           |
└──────────────────────────┬───────────────────────────────┘
                           ↓
┌──────────────────────────────────────────────────────────┐
| 9. OUTPUT (Driver)                                       |
|    - Return result to user or write to storage           |
|    - Job completes                                       |
└──────────────────────────────────────────────────────────┘
```

6.2 Shuffle: The Expensive Operation

Shuffle: Redistribute data by key across network

```
Stage 0 (Map):
Executor 0: Partition A → Keys [a, b, c]
Executor 1: Partition B → Keys [x, y, z]

        [SHUFFLE BOUNDARY]
        (Network communication)

Stage 1 (Reduce):
Executor 0: Keys [a, x, m, ...] ← All 'a's, all 'x's collected
Executor 1: Keys [b, y, n, ...] ← All 'b's, all 'y's collected
```

Shuffle Operations Trigger Network I/O:

- `reduceByKey()`
- `groupBy()`
- `join()`
- `repartition()`
- `sortBy()`
- `distinct()`

Shuffle Process:

```
1. Map Phase:
   - Each task writes intermediate results to disk
   - Partitioned by key (hash % num_partitions)
   - File name: shuffle_{job_id}_{stage_id}_{task_id}

2. Shuffle Phase:
   - Fetch: Each downstream task fetches its keys over network
   - Sort/Merge: Combine fetched data
   - Buffer: Keep in memory until full (then spill)

3. Reduce Phase:
   - Aggregate/Join/Sort: Process grouped data

Total Time = Write + Fetch + Sort + Aggregate
```

Your Project's Benchmark Impact:

- Small jobs (2-10s): Minimal shuffle (word count)
- Medium jobs (15-45s): Moderate shuffle (join)
- Large jobs (60-120s): Heavy shuffle (PageRank iterative)

Shuffle time: 20-40% of total job time

6.3 Failure & Recovery

Executor Failure:

```
Executor crashes (OOM, hardware failure, network partition)
    ↓
Driver detects heartbeat timeout (> 60s)
    ↓
Mark executor as dead
    ↓
For each task running on dead executor:
    - Identify input RDDs/partitions
    - Check if output cached elsewhere
    - If cached: Reuse ✓
    - If not cached: Recompute from lineage
       └─ Trace back to source data
       └─ Replay all transformations
       └─ Regenerate lost partition
    ↓
Reassign task to alive executor
    ↓
Continue job
```

Recovery Example:

```
// Task 5 (Stage 1) lost due to executor crash

Task 5 depends on:
  RDD_Stage1_Input ← Generated by Stage 0 Task 5
    ↑ depends on
  RDD_Stage0_Input ← Generated by Stage -1 (read)
    ↑ depends on
  CSV File on HDFS

// Recovery plan:
1. Re-read CSV file (idempotent)
2. Re-run Stage 0 Task 5 (deterministic function)
3. Re-run Stage 1 Task 5 (on different executor)
4. Continue
```

Lineage Storage: Metadata only (~100 bytes per task)

Cost: Recomputation time (1-10x job time depending on where failure occurs)

---

7. Key Configuration Parameters

7.1 Memory Configuration

```
spark = SparkSession.builder \
    .config("spark.driver.memory", "2g")         # Driver JVM h
    .config("spark.executor.memory", "4g")       # Executor JVM
    .config("spark.memory.fraction", "0.6")      # Memory for e
    .config("spark.memory.storageFraction", "0.5") # Storage /
    .config("spark.executor.memoryOverhead", "384m") # Off-hea
    .getOrCreate()
```

Your Project Tier Configuration:

```
# Small job tier
spark.config("spark.executor.memory", "512m")
spark.config("spark.executor.cores", "1")

# Medium job tier
spark.config("spark.executor.memory", "1g")
spark.config("spark.executor.cores", "2")

# Large job tier
spark.config("spark.executor.memory", "2g")
spark.config("spark.executor.cores", "2")
```

7.2 Parallelism Configuration

```
# Number of partitions for shuffle/broadcast
.config("spark.sql.shuffle.partitions", "200")  # Default: 200
.config("spark.default.parallelism", "200")     # RDD default

# High parallelism = More tasks, better load balancing, more o
# Low parallelism = Fewer tasks, less overhead, worse load bal
# Typical: 2-4x number of cores in cluster
```

Your Project: 4, 8, 16 partitions for different job sizes (good heuristic)

7.3 Scheduling Configuration

```
# FIFO (default)
.config("spark.scheduler.mode", "FIFO")

# FAIR with pools (your project's alternative)
.config("spark.scheduler.mode", "FAIR")
.config("spark.scheduler.allocation.file", "fairscheduler.xml"

# Speculation (restart slow tasks)
.config("spark.speculation", "true")
.config("spark.speculation.interval", "100ms")
.config("spark.speculation.multiplier", "1.5")
.config("spark.speculation.quantile", "0.75")
```

Explanation of Speculation:

- If task takes 1.5× (multiplier) longer than median (quantile), restart it
- Works well for stragglers
- Wastes resources for naturally slow tasks

7.4 Serialization & Compression

```
.config("spark.serializer", "org.apache.spark.serializer.KryoS
# Default: JavaSerializer (slow, large)
# Kryo: Fast, compact (2-10× speedup)

.config("spark.io.compression.codec", "snappy")
# Default: snappy
# Options: snappy (fast), lz4 (faster), gzip (better compressi

# Shuffle compression
.config("spark.shuffle.compress", "true")
.config("spark.shuffle.spill.compress", "true")
```

Impact: 20-50% reduction in network bandwidth for shuffle

---

8. Performance Considerations

8.1 Common Performance Issues & Solutions

Issue	Symptom	Cause	Solution
High GC Time	Tasks slow, application stalls	Too many objects, small heap	Increase `spark.executor.memory`, use G1GC
Shuffle Bottleneck	Stage 1 slow, high network I/O	Many partitions, unbalanced keys	Reduce partitions, repartition by distribution
Task Skew	Some tasks finish, others still running	Uneven data distribution	Use more partitions, pre-shuffle balancing
Driver OOM	Driver crashes when collecting results	Collecting too much data to driver	Use `saveAsParquet()` instead of `collect()`

Data Locality Loss	Executors assigned remote data	Executors removed/restarted	Preserve partition locality, use `coalesce()`
Slow Reads	Read stage slow	Small block size, remote storage	Increase block size to 128MB, use local storage
Shuffle Spill	Disk thrashing, job slow	Partitions too large for memory	Increase `spark.sql.shuffle.partitions`

8.2 Optimization Strategies

1. Reduce Network I/O

```
# Before (bad): Unnecessary columns read
df = spark.read.parquet("huge_dataset.parquet")
result = df.select("name", "age").show()  # Reads ALL columns,

# After (good): Column pruning + predicate pushdown
df = spark.read.parquet("huge_dataset.parquet")
result = df.select("name", "age").filter("age > 30").show()
# Spark reads only 2 columns + filters at read time
```

2. Reduce Shuffle

```
# Before: Unnecessary shuffle
df.repartition(1000)
df.groupBy("key").count()  # Shuffles to 1000 partitions

# After: Right-size partitions
num_partitions = sc.defaultParallelism * 3  # Heuristic
df.repartition(num_partitions)
df.groupBy("key").count()  # Shuffle optimized
```

3. Cache Smart

```python
# Before: Cache too much
df = spark.read.csv("data.csv")
df.cache()
df.filter("x > 10").count()  # First action triggers cache

# Problem: Cache persists filtered data, then filter again = w

# After: Cache only reused data
df = spark.read.csv("data.csv")
df_filtered = df.filter("x > 10")
df_filtered.cache()  # Cache filtered data
result1 = df_filtered.count()
result2 = df_filtered.show()  # Reuses cached filtered data
```

4. Avoid Wide Transformations

```python
# Before: Multiple shuffles
df = df.groupBy("dept").sum()
df = df.groupBy("region").avg()  # Second shuffle!
result = df.show()

# After: Single multi-aggregation
from pyspark.sql.functions import sum, avg, col
result = df.groupBy("dept", "region") \
    .agg(sum("salary"), avg("bonus")) \
    .show()  # Single shuffle!
```

8.3 Benchmarking Your Project

Your Project's Performance Characteristics:

```
Small Job (2-10s):
├─ Read: 1-2s
├─ Filter: 0.5s
└─ Collect: 0.5-7s
Total: 2-10s ✓

Medium Job (15-45s):
├─ Read: 2-5s
├─ Filter: 1-2s
├─ Join (SHUFFLE): 5-20s ← Bottleneck
├─ Aggregate: 2-5s
└─ Collect: 1-5s
Total: 15-45s ✓

Large Job (60-120s):
├─ Read: 5-10s
├─ Filter: 2-3s
├─ Repartition (SHUFFLE): 10-30s ← Bottleneck
├─ Iterative Compute: 20-50s
├─ Final Aggregate: 5-10s
└─ Collect: 5-10s
Total: 60-120s ✓
```

ML Prediction Leverage:

- Accurately predicting 15-120s jobs is **easier** than 1-5s jobs (less variance)
- Your project achieves ~85-92% accuracy on medium/large (good!)
- Small jobs have high variance (prediction harder)

---

9. Debugging & Monitoring

9.1 Spark UI

Location: `http://localhost:4040` (single application)

Key Tabs:

Tab	Shows	Useful For
Jobs	Job ID, stages, duration, status	Identify slow jobs
Stages	Stage ID, tasks, duration, shuffle I/O	Identify bottleneck stages
Tasks	Task ID, duration, locality, executor	Identify task skew, stragglers

Storage	Cached RDDs, size, location	Understand cache usage
Environment	Config parameters, memory allocation	Debug misconfiguration

9.2 Logs

Driver Log:

```
# Captured in Spark UI
# Contains: DAG compilation, stage submission, exceptions
# Look for: ERROR, WARN, Task failures

# Or save to file:
spark-submit \
  --driver-java-options "-Dlog4j.configuration=file:log4j.prop
  application.jar
```

Executor Log:

```
# On executor machine
$SPARK_HOME/work/app-*/executor-*/stdout
$SPARK_HOME/work/app-*/executor-*/stderr
```

9.3 Metrics & Instrumentation

Built-in Metrics:

```
# Runtime statistics
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()
df = spark.read.csv("data.csv", header=True)

# Get execution metrics
df.explain(extended=True)  # Show physical plan + statistics

# Get lineage
df.show()  # Then check Spark UI → DAG visualization
```

Your Project's Metrics Collection:

```python
# From your workload_metrics_collector.py
metrics = {
    "num_tasks": num_tasks,
    "num_partitions": num_partitions,
    "input_size_mb": input_size_mb,
    "peak_memory_mb": peak_memory_mb,
    "execution_time_sec": execution_time_sec,
    "shuffle_bytes_read": shuffle_bytes_read,
    "shuffle_bytes_written": shuffle_bytes_written
}
# Perfect for ML feature engineering!
```

Summary: Spark Execution Model

Single Sentence Per Component:

1. **SparkSession:** Unified entry point for Spark applications
2. **Driver:** Orchestrates DAG compilation, scheduling, result collection
3. **Executors:** Execute tasks in parallel, manage memory, store results
4. **DAG Scheduler:** Builds logical execution plan, creates stages
5. **Task Scheduler:** Assigns tasks to executors, handles retries
6. **RDD:** Immutable, distributed, resilient data structure with lineage
7. **DataFrame:** RDD with schema, optimized by Catalyst
8. **Catalyst:** Query optimizer (predicate pushdown, projection pruning)
9. **Shuffle:** Expensive data redistribution by key across network
10. **Lineage:** Metadata chain enabling failure recovery via recomputation

Key Takeaway:

> Spark is a **lazy, distributed, optimized** computation engine. User writes high-level transformations; Spark builds DAG, optimizes, partitions, schedules, executes in parallel, and recovers from failures automatically.

Your Project's Leverage:

✓ **Understands scheduler bottleneck** (FIFO head-of-line blocking) ✓ **Applies SJF scheduling** (optimal for latency by queueing theory) ✓ **Uses ML prediction** (learns job duration from features) ✓ **Respects Spark constraints** (restarts SparkContext for config changes) ✓ **Measures accurately** (collects fine-grained metrics)

Result: **25-30% speedup** on heterogeneous workloads ✓

References & Further Reading

Official Documentation

- Apache Spark Official Docs: https://spark.apache.org/docs/latest/
- Job Scheduling: https://spark.apache.org/docs/latest/job-scheduling.html
- Configuration: https://spark.apache.org/docs/latest/configuration.html

Research Papers

1. **Zaharia et al., 2010.** "Spark: Cluster Computing with Working Sets" — OSDI 2010
2. **Zaharia et al., 2008.** "Improving MapReduce Performance in Heterogeneous Environments" — OSDI 2008
3. **Thusoo et al., 2009.** "Hive: A Warehouse for Hadoop" — VLDB 2009

Books

- "Learning Spark: Lightning-Fast Big Data Analysis" — Chambers & Zaharia
- "Advanced Analytics with Spark" — Ryza, Laserson, Owen, Wiesel

Visualizations

- Spark Visualization Tools: https://databricks.com/blog/spark-visualization
- DAG Visualization: Built into Spark UI

End of Document

Appendix: Quick Reference — Spark Configuration Checklist

```python
# Development Setup
spark = SparkSession.builder \
    .master("local[*]") \
    .appName("MyApp") \
    .config("spark.driver.memory", "2g") \
    .config("spark.executor.memory", "2g") \
    .config("spark.sql.shuffle.partitions", "4") \
    .getOrCreate()

# Production Setup
spark = SparkSession.builder \
    .master("yarn") \
    .appName("MyApp") \
    .config("spark.executor.memory", "4g") \
    .config("spark.executor.cores", "4") \
    .config("spark.sql.shuffle.partitions", "200") \
    .config("spark.serializer", "org.apache.spark.serializer.K
    .config("spark.scheduler.mode", "FAIR") \
    .getOrCreate()

# Your Project Setup
spark = SparkSession.builder \
    .master("local[*]") \
    .appName("BDA-Lab-Adaptive-Scheduler") \
    .config("spark.driver.memory", "1g") \
    .config("spark.executor.memory", "512m")  # or 1g/2g by ti
    .config("spark.executor.cores", "1")  # or 2 by tier \
    .config("spark.sql.shuffle.partitions", "4")  # or 8/16 by
    .config("spark.scheduler.mode", "FAIR") \
    .config("spark.scheduler.allocation.file", "fairscheduler.
    .getOrCreate()
```